🏍 Bike Troubleshooting Bot

A grounded, multilingual RAG assistant for motorcycle manuals — built on Sarvam AI

Sarvam-30B | Sarvam Vision | RAG + FAISS | English + Hindi | Streamlit

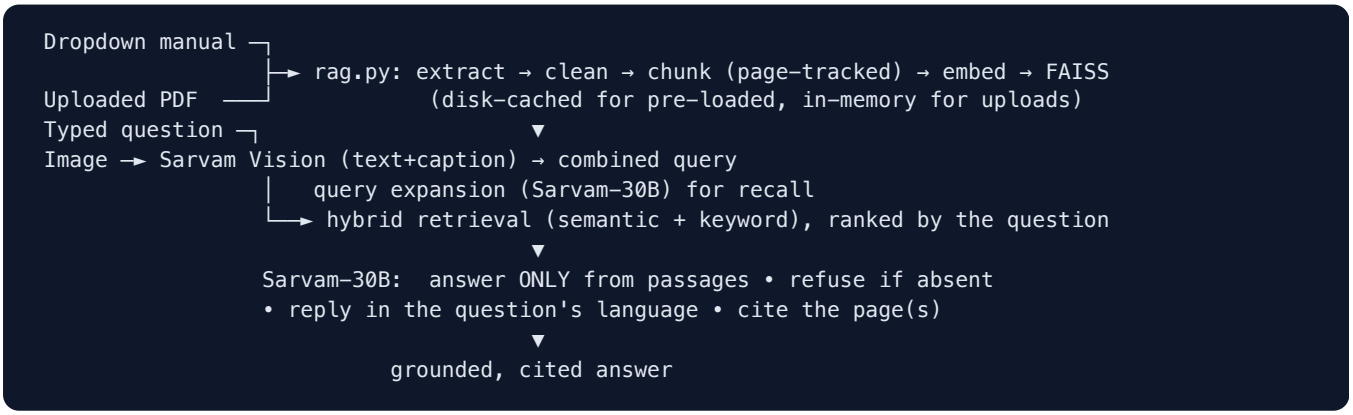
Live app: https://bike-troubleshooting-bot-rcmxvm5w8rh897xbyoxmne.streamlit.app/
GitHub: https://github.com/utkarshgupta-sketch/bike-troubleshooting-bot

A Streamlit web app that answers motorcycle questions **strictly from the bike's official manual** — in **English or Hindi**, from **typed questions and/or photos** — and **politely refuses** when the answer isn't in the manual. The defining behaviour is **document-grounded answering with reliable refusal**: the bot never invents an answer; it uses only the retrieved manual passages and cites the page(s) it used.

What it does

- **Pick a manual** from a dropdown (auto-built by scanning the `manuals/` folder) **or upload your own PDF** (used in-session only — never saved, catalogued, or logged).
- **Ask by text, image, or both.** A photo (dashboard light, warning label, a part, a manual page) is read by Sarvam Vision (text + a description) and merged into the question.
- **Grounded, cited answers** — only from the manual, with page citations.
- **Rephrase-before-refuse.** On a miss it first asks you to reword; only if that also fails does it give the firm "consult a qualified mechanic or your dealer" refusal.
- **Multilingual.** Ask in English or Hindi; the answer returns in the **language of your question**, even if the manual is in the other language.

How it works (the pipeline)



Key design choices

- **Grounded refusal** — a strict system prompt; the model tags replies `NOT_FOUND` (detected language-independently), driving the rephrase-then-refuse flow.

- **Hybrid retrieval** — semantic similarity *plus* literal keyword matching, so exact terms (e.g. “exhaust”) are reliably found.
- **Query expansion** — the question is rewritten into several search angles for recall, but candidates are **ranked by the original question** for precision.
- **Honest page citations** — detect the manual's **printed** page number (handling cover offsets and doubled-number OCR quirks); else label “document page N”.
- **OCR rescue** — manuals whose PDFs have no text layer are OCR'd once with Sarvam Vision and cached, never re-paid for.

Tech stack

Layer	Choice
UI	Streamlit (Python 3.11+)
Answers + query expansion	**Sarvam-30B** via `sarvamai` (not legacy Sarvam-M)
Image understanding + OCR	**Sarvam Vision** (Document Intelligence)
PDF text extraction	PyPDF2 (fast, free, for digital-native manuals)
Embeddings	sentence-transformers `paraphrase-multilingual-MiniLM-L12-v2` (local)
Vector store	FAISS (`faiss-cpu` , in-process)
Language detection	langdetect

Multilingual support

Works end-to-end in **English and Hindi**, including the **cross-language** case (English manual + Hindi question → Hindi answer), thanks to the multilingual embedder and Sarvam-30B. Extensible by adding a manual with the right `_<Language>` filename suffix — no code changes.

Building this — notes from the build

Sarvam models used

- **Sarvam-30B** — all chat: grounded answers, refusal logic, query expansion.
- **Sarvam Vision** — image understanding (text + caption) and the one-time OCR of the Hindi manual, whose PDF had no extractable text layer.
- Embeddings are **local** (sentence-transformers) — retrieval is free and offline.

Cost to develop

≈ **₹130 of Sarvam credit** (of ₹600 available). The bulk was the **one-time OCR of the 161-page Hindi manual (~₹90)** at ₹5 per 10-page Vision job — cached so it's never re-paid. The rest was Vision

image/table experiments (~₹30) and Sarvam-30B chat (token-priced, modest). Local embeddings and FAISS cost nothing.

Challenges we hit and solved

Challenge	Solution
Hindi PDF stored text as vector outlines — PyPDF2 read nothing	OCR-rescue with Sarvam Vision, cached to disk
Exact-term queries (“exhaust”) ranked poorly by the paraphrase embedder	Hybrid semantic + keyword retrieval
Re-phrasings gave different answers (“handling is bad” vs “why is handling poor”)	Query expansion for recall, ranked by the original question
Citations off by N pages (physical vs printed; doubled OCR numbers)	Printed-page detection with honest fallbacks
Reasoning consumed the 4096-token tier cap (empty answers) / repetition loops	Shorten-and-retry, frequency penalty, robust `NOT_FOUND` parsing
422 context overflow — Vision embeds the image as base64 in OCR output	Strip embeds + clean the cached file
Image + Hindi question flipped the answer language & derailed retrieval	Rank & detect language from the typed question; image as context
Cloud cold-starts & variable API latency	Prebuilt indexes, model pre-warm, capped CPU threads, timeouts, “Faster answers”

How the thinking evolved

We started **plan-first** (no code until the plan was approved) and stayed honest throughout — verifying API details against the docs rather than guessing, and **correcting our own wrong assumptions** when the evidence disagreed. For example, we first concluded Sarvam Vision couldn't caption images and reached for Gemini; on closer inspection Sarvam *does* caption natively, so we reverted to a pure-Sarvam stack. The recurring lesson: the hard cases (terse table answers) are a **retrieval ceiling**, not a prompt bug — so we invested in robustness (hybrid retrieval, rephrase-before-refuse) and **documented the limits** rather than papering over them.

Built with Claude Code. The entire app was vibe-coded collaboratively in a single, continuous Claude Code session — plan, build, debug, and deploy. That one session grew to roughly **~557K tokens of context**, holding the whole journey in working memory (the original brief, every design decision, and the full bug-fix trail) — which is what made the one-session, plan-first workflow possible.

Known limitations & honest notes

- **Tables (maintenance/spec schedules).** PyPDF2 flattens 2-D tables; Sarvam Vision parses them into structured HTML (verified). We flag this rather than routing all extraction through Vision (cost/latency).

- **English vs Hindi manual can differ** on table facts — separate PDFs, different extraction; the bot is faithful to what it retrieves (not a mistranslation).
- **Retrieval variance** on terse table/checklist answers — cushioned by rephrase-before-refuse.
- **Image input reads text + describes; it doesn't diagnose.**
- **Uploaded manuals are session-only.**
- **Response time depends on Sarvam API load** (~20–60s typical, more on spikes); handled with staged progress, timeouts, and a "⚡ Faster answers" toggle.

Built with the Sarvam AI platform (Sarvam-30B, Sarvam Vision) and open-source tooling (Streamlit, sentence-transformers, FAISS, PyPDF2). Developed with Claude Code.